

Assignment 1a, Number fiddling

In this assignment you will implement functions that handle integers and can be used for "number fiddling".

These functions will be used in assignment 1b where you will implement a game called "Bulls and cows". Make sure that your functions work before starting part 1b by using the tests that you get in the source code for this assignment.

In this assignment you should use: `int`'s, `if`, `while` and functions (i.e., no `switch`, no `for`-loops, no arrays, no `string` or other more advanced features)!

Functions to implement

Implement the following functions:

`n_digit`

- Count the number of digits in an integer number, e.g., for the number 45 the function should return 2.

`get_digit_at_position`

- Return the digit at a given position in an integer number.
- The position should be given from the right and start with zero, e.g., for the number 75934 the digit at position 0 is 4 and the digit at position 2 is 9.

`digit_is_in`

- Check if a number contains a certain digit, e.g, for the number 7645 the function should return `True` if the digit is 6 while it should return `False` if the digit is 8.

`is_unique_digits`

- Check if all digits in a number are unique, i.e., that there are no repeated digits in the number. For the number 5863, e.g., the function should return `True` while for the number 56786 the function should return `False`.
- Try to avoid excessive "if" statements in this function.

`get_random_digits`

- Return a number with unique random digits.
- The function should be able to provide a number with arbitrary many random digits, i.e., the number of random digits is given as an argument to the function.
- All digits in the number should be unique, i.e., there are no repeated digits in the returned number.
- The digits should be in the range 1-9, i.e., the number should not include 0 (to make the function more useful in assignment 1b).
- The function should use `rand()`, but the initialization of `rand()` should be made in main.

- You should choose a suitable maximum value for the number of digits (based on the requirements above).
- Make sure that you get different random digits each time you run the code.

Hint:

- Use "brute force", i.e., generate a new random number until it fulfills the requirements.

`n_exact_digit_matches`

- Count exact digit matches between two numbers (larger than 0).
- An exact match is when the same digit is at the same position in both numbers.
- You may assume that the two numbers have the same number of digits.

`n_digit_matches`

- Count digit matches between two numbers with unique digits.
- A match is when a digit is in both numbers but not necessarily at the same position.
- You can assume that the function will be used only for unique digits.

Comments

Note that the functions should not only work for the examples given above, they should work for general cases!

Implement functions from top to bottom and test each function before moving on to the next function.

You are free to introduce more functions that can be used within the functions above (for example the `pow_int` function from set2).

Hints

- Think about what features the decimal number system has and how these features can be exploited together with modulo and integer division.
- See 'samanfattningar/rand.pdf' for a summary of how `rand()` works.

Useful exercises for this assignment:

set2/ex_functions

Useful example files for inspiration:

set1/sample_random: for demonstration of `rand()`

set1/sample_operators: for demonstration of integer division and modulo

set2/sample_functions: for inspiration on how to use modulo

Reporting the assignment

To pass the assignment you have to show one of the teachers that your functions pass all tests. We will also ask you to explain part of the code so be prepared to show that you know how it works.